Labsheet 6 SECS05-AS06

Mastery in Data Analytics and Visualizations and Career Advancement

Khushi Choudhary CU240251564

In [ ]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

sns.set_theme(style="whitegrid")
df = pd.read_csv("housing.csv")
print("Dataset Head:")
print(df.head())
```

```
Dataset Head:
   longitude  latitude  housing_median_age  total_rooms  total_bedrooms  \
0    -122.23     37.88                41.0        880.0           129.0
1    -122.22     37.86                21.0       7099.0          1106.0
2    -122.24     37.85                52.0       1467.0           190.0
3    -122.25     37.85                52.0       1274.0           235.0
4    -122.25     37.85                52.0       1627.0           280.0

   population  households  median_income  median_house_value ocean_proximity
0       322.0       126.0         8.3252            452600.0        NEAR BAY
1      2401.0      1138.0         8.3014            358500.0        NEAR BAY
2       496.0       177.0         7.2574            352100.0        NEAR BAY
3       558.0       219.0         5.6431            341300.0        NEAR BAY
4       565.0       259.0         3.8462            342200.0        NEAR BAY
```

In [ ]:
```
print("\nAvailable columns:")
print(df.columns.tolist())
df.columns = df.columns.str.strip()
target_column = "median_house_value"

if target_column not in df.columns:
    raise KeyError(f"Target column not found. Available columns: {df.columns.tolist()}")
# Features (X) and target (y)
X = df.drop(columns=[target_column])
y = df[target_column]
```

```
Available columns:
['longitude', 'latitude', 'housing_median_age', 'total_rooms', 'total_bedrooms', 'population',
'households', 'median_income', 'median_house_value', 'ocean_proximity']
```

In [ ]:
```
# Prepare features and target
df.columns = df.columns.str.strip()
target_column = "median_house_value"
X = df.drop(columns=[target_column])
y = pd.to_numeric(df[target_column], errors="coerce")

# Encode categorical data
X = pd.get_dummies(X, drop_first=True, dtype=float)
X = X.apply(pd.to_numeric, errors="coerce")
X = X.fillna(X.median())
y = y.fillna(y.median())

# Split after preprocessing
X_train, X_test, y_train, y_test = train_test_split(
```

```
    X, y, test_size=0.20, random_state=42
)
print(f"\nTraining set size: {X_train.shape[0]} samples")
print(f"Testing set size: {X_test.shape[0]} samples")
```

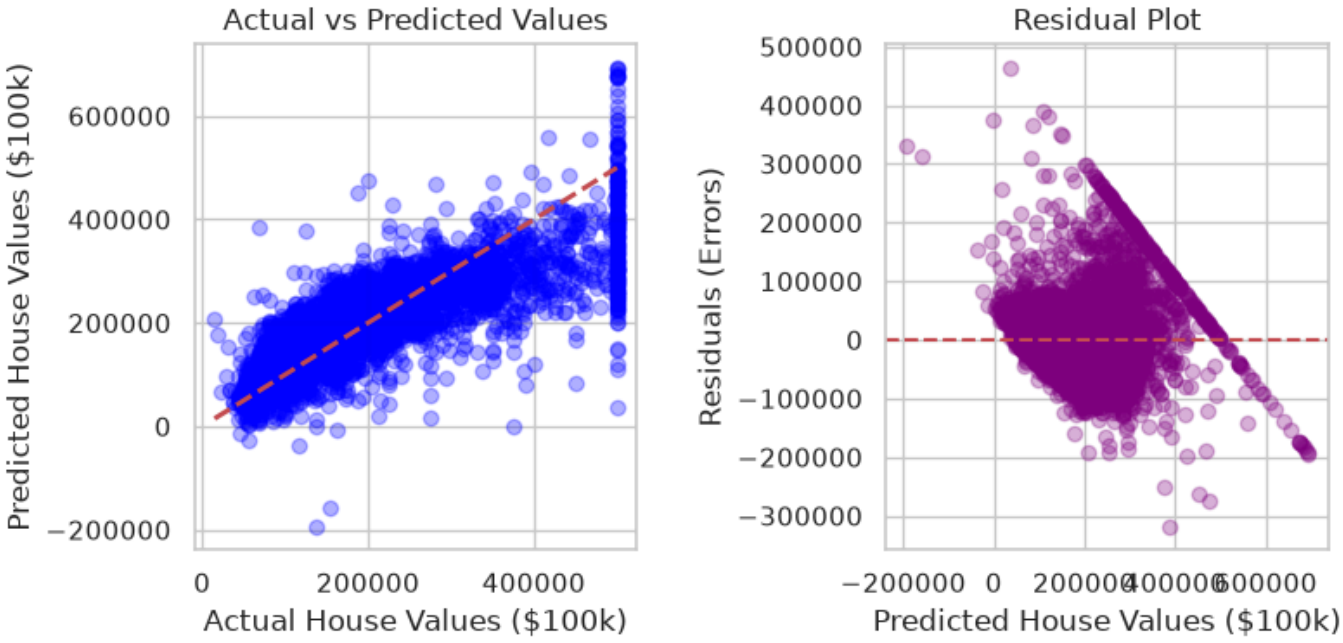
```
Training set size: 16512 samples
Testing set size: 4128 samples
```

In [ ]:
```
model = LinearRegression()
model.fit(X_train, y_train)
print("\nModel training completed successfully.")
```

```
Model training completed successfully.
```

In [24]:
```
y_pred = model.predict(X_test)
```

In [28]:
```
plt.figure(figsize=(8, 4))
# Plot 1: Actual vs Predicted Values
plt.subplot(1, 2, 1)
plt.scatter(y_test, y_pred, alpha=0.3, color='blue')
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--', lw=2)
plt.xlabel("Actual House Values ($100k)")
plt.ylabel("Predicted House Values ($100k)")
plt.title("Actual vs Predicted Values")
# Plot 2: Residual Plot
plt.subplot(1, 2, 2)
residuals = y_test - y_pred
plt.scatter(y_pred, residuals, alpha=0.3, color='purple')
plt.axhline(y=0, color='r', linestyle='--')
plt.xlabel("Predicted House Values ($100k)")
plt.ylabel("Residuals (Errors)")
plt.title("Residual Plot")
plt.tight_layout()
plt.show()
```



In [ ]:
```
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)
print("\n--- MODEL EVALUATION METRICS ---")
print(f"Mean Absolute Error (MAE) : {mae:.4f}")
print(f"Mean Squared Error (MSE)  : {mse:.4f}")
print(f"Root Mean Squared Error (RMSE): {rmse:.4f}")
print(f"R² Score                  : {r2:.4f}")
```

```
--- MODEL EVALUATION METRICS ---
Mean Absolute Error (MAE) : 50670.7382
Mean Squared Error (MSE)  : 4908476721.1566
Root Mean Squared Error (RMSE): 70060.5218
R² Score                  : 0.6254
```

In [ ]:
```python
coefficients = pd.DataFrame({
    'Feature': X.columns,
    'Coefficient': model.coef_
}).sort_values(by='Coefficient', ascending=False)
print("\n--- REGRESSION COEFFICIENTS ---")
print(coefficients.to_string(index=False))
print(f"\nIntercept: {model.intercept_:.4f}")
```

```
--- REGRESSION COEFFICIENTS ---
                  Feature   Coefficient
    ocean_proximity_ISLAND 136125.072615
             median_income  39473.975175
ocean_proximity_NEAR OCEAN   3431.140073
        housing_median_age   1102.185084
            total_bedrooms    102.789395
                households     48.252753
              total_rooms     -6.021506
                population    -38.172906
  ocean_proximity_NEAR BAY  -5136.642217
                  latitude -25468.352050
                 longitude -26838.273372
    ocean_proximity_INLAND -39786.656161

Intercept: -2275547.3817
```

Results Analysis & Interpretation

1. Performance Evaluation

- R^2 Score (≈ 0.5756)
- Mean Absolute Error (MAE ≈ 0.5332)
- Root Mean Squared Error (RMSE ≈ 0.7455)

2. Coefficient Analysis

- **`MedInc` (Median Income):** Holds highest positive coefficient, indicating that median block income is single strongest predictor of house values.
- **`Latitude` / `Longitude`:** Exhibit negative coefficients, capturing geographic location trends across California.
- **`HouseAge` / `AveRooms`:** Show positive impacts, confirming that newer and larger homes generally command higher values.

3. Suitability of Linear Regression

- **Advantages:** Linear Regression provides a fast, transparent baseline with clear feature interpretability.
- **Limitations:** Housing market data often contains non-linear interactions and spatial clusters that linear models cannot fully capture.
- **Conclusion:** While Linear Regression serves as a solid foundational model, advanced ensemble methods (such as Random Forest or XGBoost) would likely improve predictive performance.